



UNIVERZITET U NOVOM SADU FAKULTET TEHNIČKIH NAUKA



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
NOVI SAD

ISPITNI RAD

Kandidat: Marko Sladojević

Broj indeksa: SV33-2023

Predmet: Paralelno Programiranje

Tema rada: 2valves – Analiza Medija u Srbiji

Mentor rada: dr Dušan Kenjić

Novi Sad, septembar 2025.

Sadržaj

1. Uvod.....	3
1.1 Kratak opis projekta	3
1.2 Tehnologije koje su korišćene	3
2. Analiza problema paralelizacije.....	4
2.1 Priroda <i>I/O-bound</i> operacija u <i>web scraping</i> -u.....	4
2.2 Specifični izazovi medijskog <i>landscape</i> -a u Srbiji.....	4
2.3 Skalabilnost i učestalost ažuriranja	4
2.4 Tehnički bottleneckovi sekvencijalnog pristupa.....	5
2.5 Opravdanje paralelnog pristupa	5
3. Koncept rešenja.....	6
3.1 Dvonivovska paralelizacija.....	6
3.2 Analiza izbora arhitekture	7
3.3 Prednosti hijerarhijskog pristupa	7
3.4 Optimizacije i thread safety	8
4. Arhitektura projekta.....	9
5. Finalni rezultat.....	12

1. Uvod

1.1 Kratak opis projekta

„2valves“ je aplikacija razvijena u jeziku C++, sa fokusom na paralelnost. U kom smislu? U svakom mogućem. *Web scraping* obavljamo paralelno pomoću *TBB* biblioteke, a analiziramo paralelne svetove prikazane ljudima koji prate različite medije u Srbiji. Projekat sadrži C++ skriptu koja paralelno skrepuje 2 nezavisna medija: 021.rs, N1info.rs, kao i 2 režimska: Kurir.rs i Informer.rs. Skriptu koristi *backend CRON-job* koji svakih par sati ponovo skrepuje najnovije relevantne vesti, koje se persistuju u *MongoDB* bazi podataka. *Backend* isporučuje i *endpoint* na kome je moguće za određeni dan pozvati objektivnu analizu vesti od strane *Gemini API*-ja. Šlag na torti je *Svelte front* u *neobrutalistic* stilu koji korisniku približava tu analizu.

1.2 Tehnologije koje su korišćene

1. *Intel oneTBB* – biblioteka za paralelizaciju (*parallel_for*).
2. *cpr* – fork od *libcurl*-a za *HTTP GET* zahteve i preuzimanje *HTML* sadržaja.
3. *nlohmann/json* – za generisanje JSON izlaza.
4. *CMake* – za *build* projekta.
5. *vcpkg* – *package manager*
6. *flask* – *backend* tehnologija u *python*-u
7. *svelte* – *frontend* tehnologija

8. *Gemini API* – analiza sadržaja

2. Analiza problema paralelizacije

2.1 Priroda *I/O-bound* operacija u *web scraping-u*

Web scraping je inherentno *I/O-bound* proces koji se suočava sa nekoliko ključnih *bottleneckova*:

Mrežna latencija - Svaki *HTTP* zahtev uključuje mrežno kašnjenje koje može varirati od 50ms do nekoliko sekundi, zavisno od odziva servera i kvaliteta konekcije. U sekvencijalnom pristupu, procesor ostaje neiskorišćen dok čeka odgovor servera.

Čekanje na *server response* - Medijski sajtovi često implementiraju rate limiting ili mogu imati sporije vreme odgovora tokom vršnih sati, što dodatno produžava vreme izvršenja sekvencijalnog pristupa.

Neiskorišćeni *CPU* resursi - Dok se čeka mrežni *I/O*, *CPU* kapacitet ostaje neiskorišćen, što predstavlja značajan gubitak performansi, posebno na multi-core sistemima.

2.2 Specifični izazovi medijskog *landscape-a* u Srbiji

Različite arhitekture sajtova - Svaki od četiri sajta (021.rs, N1info.rs, Kurir.rs, Informer.rs) ima jedinstvenu *DOM* strukturu i različite pristupe učitavanju sadržaja, što zahteva fleksibilnu paralelnu arhitekturu.

Varijabilno vreme odgovora - Ralličiti mediji mogu imati značajno različita vremena odziva, što čini sekvencijalni pristup još manje efikasnim jer najsporiji sajt određuje ukupno vreme izvršenja.

Dinamički sadržaj - Mnogi sajtovi koriste JavaScript za dinamičko učitavanje sadržaja, što zahteva dodatno vreme čekanja koje se u sekvencijalnom pristupu sabira linearno.

2.3 Skalabilnost i učestalost ažuriranja

***CRON* job frekvencija** - Potreba za redovnim ažuriranjem (svakih par sati) zahteva da proces scrapinga bude što kraći kako bi se smanjilo opterećenje servera i omogućila veća frekvencija ažuriranja.

Rastući broj izvora - Sekvencijalni pristup postaje eksponencijalno sporiji sa dodavanjem novih medijskih izvora, dok paralelni pristup omogućava linearno skaliranje.

2.4 Tehnički bottleneckovi sekvencijalnog pristupa

Linearno sabiranje latencije - U sekvencijalnom modelu, ukupno vreme = $\Sigma(\text{vreme_čekanja}_i + \text{vreme_obrade}_i)$, što može rezultovati minutama čekanja za obradu sva četiri sajta.

Neoptimalno korišćenje memorije - Sekvencijalno učitavanje i obrada može dovesti do fragmentacije memorije i suboptimalnog cachinga.

Blokiranje na najsporijim resursima - Jedan spor sajt može značajno usporiti ceo proces, što je posebno problematično u realnom vremenu.

2.5 Opravdanje paralelnog pristupa

TBB (Threading Building Blocks) biblioteka omogućava:

- **Concurrency na task nivou** - Simultano scrapovanje sva četiri sajta
- **Optimalno iskorišćenje CPU** - Dok jedan thread čeka *I/O*, drugi mogu obrađivati podatke
- **Fault isolation** - Greška na jednom sajtu ne blokira ostale
- **Load balancing** - *TBB* automatski distribuira zadatke između dostupnih threadova

Rezultat je dramatično smanjenje ukupnog vremena izvršenja sa potencijalnog poboljšanja reda veličine, što direktno utiče na korisnost i skalabilnost sistema za analizu medijskog sadržaja u realnom vremenu.

3. Koncept rešenja

Rešenje implementira **hijerarhijski paralelizam** koristeći ugnježdene `parallel_for` konstrukcije iz TBB biblioteke, što omogućava optimalno iskorišćenje resursa na dva nivoa granularnosti.

3.1 Dvonivovska paralelizacija

Spoljašnji `parallel_for` - paralelizacija po izvorima:

```
tbb::parallel_for(tbb::blocked_range<size_t>(0, scrapers.size()),
    [&](const tbb::blocked_range<size_t>& range) {
        // Obradjuje svaki od 4 medijska izvora paralelno
    }
);
```

Ovaj nivo omogućava simultanu obradu sva četiri medijska sajta (Informer.rs, Kurir.rs, 021.rs, N1info.rs). Svaki thread dobija zaduženje za kompletan workflow jednog sajta - od dohvaćanja liste linkova do skladištenja svih članaka.

Unutrašnji `parallel_for` - paralelizacija po člancima:

```
tbb::parallel_for(tbb::blocked_range<size_t>(0, links.size()),
    [&](const tbb::blocked_range<size_t>& linkRange) {
        // Svaki link se obrađuje paralelno
        auto article = processUrl(scraperType, links[j]);
    }
);
```

3.2 Analiza izbora arhitekture

Zašto ne `parallel_pipeline`?

Pipeline model bi imao sledeće faze:

1. **Generisanje *URL*-ova** - relativno brza operacija
2. **Preuzimanje stranica** - *I/O-bound*, glavni *bottleneck*
3. **Parsiranje sadržaja** - *CPU-bound*, ali brža od preuzimanja

Problem sa pipeline pristupom u ovom slučaju:

- **Nebalansiranost faza** - faza preuzimanja je značajno sporija od ostalih
- ***Overhead* sinhronizacije** - dodatno vreme potrebno za koordinaciju između faza
- **Suboptimalna iskorišćenost** - *CPU-bound* faze bi čekale na *I/O-bound* fazu

3.3 Prednosti hijerarhijskog pristupa

Maksimalno iskorišćenje *I/O* paralelizma: Umesto čekanja da se jedan članak preuzme pre prelaska na sledeći, sistem simultano šalje *HTTP* zahteve za multiple članke sa svih sajtova. Ovo direktno napada glavni *bottleneck* - mrežnu latenciju.

Prirodan *load balancing*: *TBB* automatski distribuira članke između dostupnih *threadova*. Ako jedan sajt ima sporije odgovore, ostali *threadovi* mogu nastaviti sa obradom članaka sa drugih sajtova.

Fault isolation:

```
tbb::concurrent_vector<json> results; // Thread-safe kontejner
```

Greška pri *scrapovanju* jednog članka ili sajta ne utiče na ostale operacije. *concurrent_vector* omogućava *thread-safe* dodavanje rezultata bez eksplicitne sinhronizacije.

3.4 Optimizacije i thread safety

Memory efficiency:

```
std::unique_ptr<NewsScraper> scraper =  
NewsScraperFactory::createScraper(scraperType);
```

Factory pattern omogućava kreiranje specifičnog *scraper*-a za svaki sajt, sa automatskim upravljanjem memorije.

Concurrent data structures: Korišćenje *tbb::concurrent_vector* eliminiše potrebu za eksplicitnim *mutex*-ima i omogućava *lockfree* dodavanje rezultata iz različitih *threadova*.

```

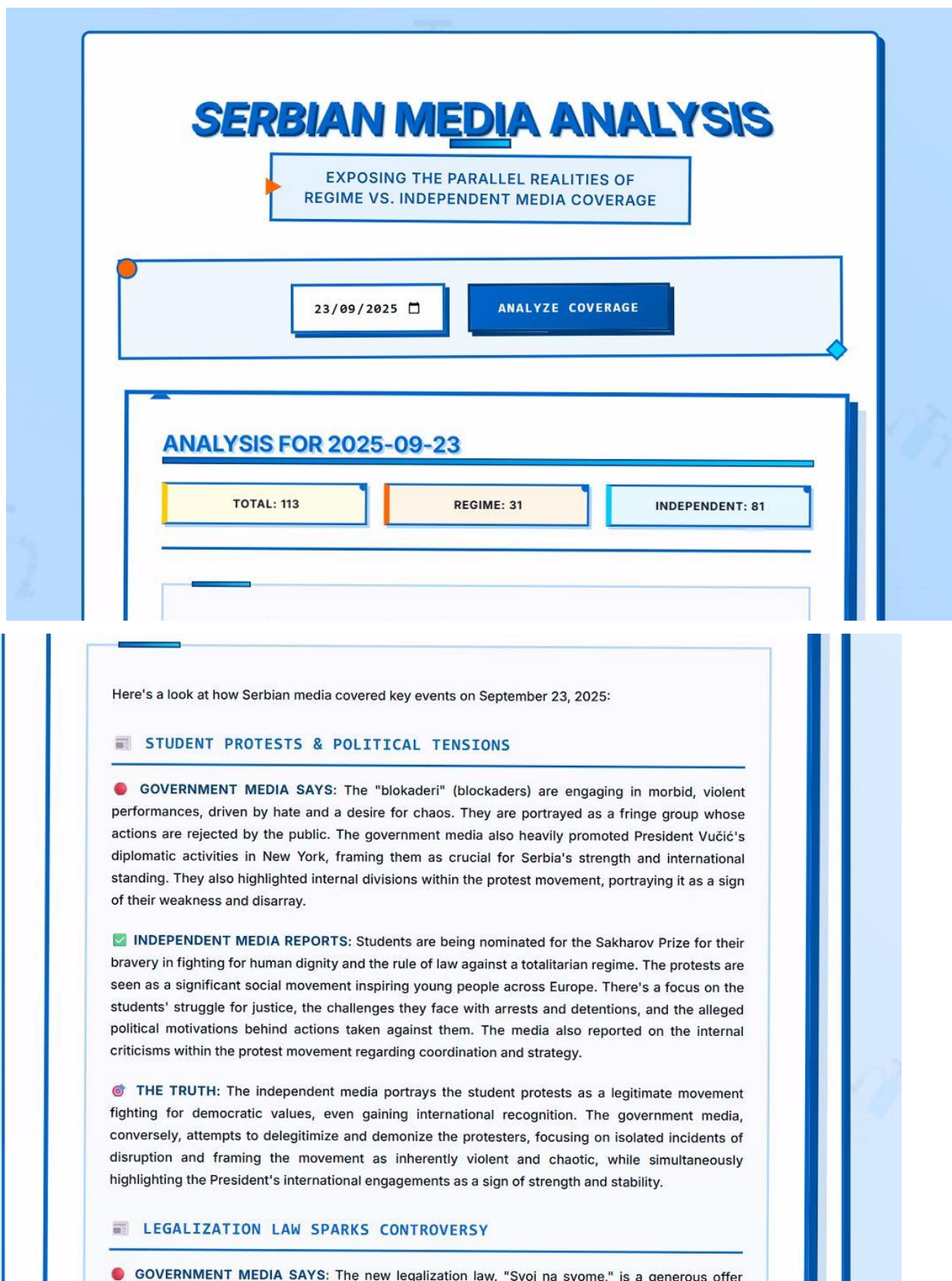
2valves/
├── folder cpp-script/ # C++ paralelni scraping engine
│   ├── file CMakeLists.txt # CMake build konfiguracija
│   ├── file vcpkg.json # Dependency management (TBB, nlohmann/json)
│   ├── file 2valves.sln # Visual Studio solution
│   └── folder 2valves/ # Main C++ projekt
│       ├── file 2valves.vcxproj # VS projekt fajl
│       ├── file 2valves.vcxproj.filters # VS folder struktura
│       ├── file 2valves.cpp # Main entry point
│       └── file ParallelWorker.cpp/.h # TBB parallel_for implementacija
│           ├── file NewsScraperFactory.cpp/.h # Factory pattern za scraper-e
│           ├── file NewsScraper.cpp/.h # Bazna klasa za sve scraper-e
│           ├── file InformerScraper.cpp/.h # Scraper za Informer.rs
│           ├── file KurirScraper.cpp/.h # Scraper za Kurir.rs
│           ├── file N1Scraper.cpp/.h # Scraper za N1info.rs
│           └── file Scraper021.cpp/.h # Scraper za 021.rs
└── folder flask-api/ # Python Flask backend
    ├── file app.py # Flask aplikacija entry point
    ├── file config.py # Konfiguracija (MongoDB, API keys)
    ├── file requirements.txt # Python dependencies
    └── folder routes/ # API endpoint definicije
        └── file __init__.py

```

			🔗	api.py	# Glavni API endpoints
			❤️	health.py	# Health check endpoints
			📁	services/	# Business logic servisi
			📄	__init__.py	
			🔧	cpp_builder.py	# C++ build i execution
			📄	database.py	# MongoDB operacije
			🤖	gemini_service.py	# Gemini AI integracija
			🕒	scheduler.py	# CRON job management
			📁	models/	# Data modeli
			📄	__init__.py	
			📊	article_processor.py	# Članak processing logika
			📁	utils/	# Utility funkcije
			📄	__init__.py	
			📝	logging_config.py	# Logging konfiguracija
			📁	frontend/	# Svelte frontend
			📄	package.json	# Node.js dependencies
			📄	package-lock.json	# Lockfile
			🌐	index.html	# Main HTML template
			⚙️	svelte.config.js	# Svelte konfiguracija
			⚡	vite.config.js	# Vite bundler config
			📁	src/	# Svelte source kod
			🎮	App.svelte	# Main Svelte komponenta
			🎨	app.css	# Neobrutalist styling
			🚀	main.js	# Svelte app bootstrap
			📁	pics/	# UI assets

```
|      └─ 🖼️ pump.png
|      └─ 🖼️ pump_transparent.png
|
└─ 📁 materials/                                # Projektna dokumentacija
    └─ 📄 PP-Specifikacija-projekta-2025.pdf
|
└─ 📄 setup.py                                  # Python package setup
└─ 📄 package-lock.json                         # Node.js global lockfile
└─ 📄 README.md                                # Projektna dokumentacija
└─ 📄 .gitignore                               # Git ignore rules
```

5. Finalni rezultat



.... and so on and on 😊